

Joint convexification: exact support, complete tolerance separation, and original-model cuts

October 2026

Abstract

Joint convexification uses the dependence among nonlinear expressions that share variables and domain constraints. This report develops the support, separation, representation, and verification steps needed to turn that principle into a specified solver component. It gives exact rational support for quadratic objectives on bounded rational polytopes, retains a polynomial support algorithm for constrained quadratic stars, and supplies a finite positive-tolerance separation procedure with explicit costs and failure statuses. An original-variable aggregation formula transfers joint support bounds to linear cuts without introducing nonlinear feature equalities. The implementation checks original expression domains, model binding, and the final floating-point row. In a new prospective 30-model application sample, native baseline and both cut modes solve the same 25 models within the 30-second budget. Cut modes use more summed time; the result does not support default activation. Native SCIP remains the default. A separate 75-job matched validation checks repairs for four retained diagnostic worker errors and discovery-budget violations, with no extra solves. All 165 recorded cuts across both campaigns pass replay; the four original missing logs remain excluded. Exact support, complete tolerance separation, numerical solver performance, and complete certification of a solve are distinct claims. The report proves and implements the first two on their stated classes and evaluates the third; it does not claim the fourth or publication priority for established convexification, duality, or quadratic-programming methods.

Contents

1	The problem and the completion standard	2
2	Joint support cuts	3
3	Finite certificates on the whole domain	4
3.1	Exact Bernstein bounds for rational polynomials	4
3.2	Supported elementary expressions and explicit failure	5
4	Joint cuts in the original variables	5
5	Exact quadratic support on a bounded polytope	7
6	A larger exact class: quadratic stars with affine rows	8
7	What overlapping pair hulls miss	10
7.1	The stronger interface that makes tree gluing valid	11

8	Complete separation to a positive tolerance	12
9	The implemented model, cut, and replay contract	14
9.1	Source arithmetic and native expression binding	14
9.2	Original affine implications and exact domains	15
9.3	Block discovery and bounded practical separation	16
9.4	Replay and the trust base	17
10	Prospective evaluation and verification	17
10.1	The earlier result is retained	17
10.2	Frozen new comparison	18
10.3	Prospective campaign results	19
10.4	Matched repair validation	20
10.5	Targeted implementation and document checks	21
11	Prior results and contribution boundary	21
12	What completion does and does not imply	22

1 The problem and the completion standard

Several functions of the same variables can be convexified together by minimizing their weighted sum over the common domain. This familiar observation leaves several practical questions. Which domains and functions have a complete support algorithm? Does a direction search actually separate a point, or merely fail to find a cut? Can the cut be applied while retaining the native nonlinear rows? Does the exact inequality survive floating-point assembly? Does the resulting solver solve more problems, or spend more time on work that its native relaxations already accomplish?

The previous repository continuation supplied certified support cuts, exact coupled-pair and constrained-star oracles, automatic graph-block discovery, and a short controlled experiment. It left native SCIP as the recommended default: among 24 selected application models, 20 passed the common importer, baseline and reformulation control solved 19, and each cut mode solved 18. It also exposed three domain-admission refusals and one unsupported variable-exponent model. These findings motivate the present work; they are not silently replaced by its new experiment. The original report and raw results remain in [research-20261002-convexification/document/main.pdf](#) and [the first campaign records](#).

Completion has a concrete meaning here. The supported algorithms must have explicit inputs and statuses; proofs must cover their complete admitted domains, including degeneracy; returned certificates must bind the actual model and row; a complete separation mode must distinguish a checked positive-tolerance result from an exhausted heuristic; and empirical claims must match a frozen, reproducible comparison. A theorem that an arbitrary nonlinear problem can be solved efficiently is not part of that standard. Section 12 proves an obstruction to that stronger interpretation rather than treating it as an omitted implementation step.

The mathematical chain is self-contained. Sections 2 and 3 restate the inherited support and certification foundation. Sections 4–8 develop the current extensions and retain the constrained-star and overlap results needed to understand their scope. Section 9 binds those results to the actual implementation. The [claim and evidence map](#) distinguishes proved results, implemented APIs, internal review, numerical checks, and measured outcomes. Internal review is not formal verification or journal peer review.

2 Joint support cuts

Let $D \subset \mathbb{R}^n$ be nonempty and compact, and let $F = (f_1, \dots, f_k) : D \rightarrow \mathbb{R}^k$ be continuous. Write

$$G_D = \{(x, F(x)) : x \in D\}, \quad H_D = \text{conv } G_D.$$

The lifted variables w_j represent the *same* functions f_j of the *same* variables x . For a direction $d = (a, b) \in \mathbb{R}^{n+k}$ define

$$h_D(a, b) = \min_{x \in D} \{a^\top x + b^\top F(x)\}. \quad (1)$$

We use lower support values, so every cut is written with a lower bound. If integer variables occur in D , support over a continuous superset remains valid for the mixed-integer graph, but need not describe its hull.

Proposition 2.1 (Support and complete hull description). *If $\underline{h} \leq h_D(a, b)$, then*

$$a^\top x + b^\top w \geq \underline{h} \quad (2)$$

is valid for H_D . Moreover,

$$H_D = \bigcap_{(a,b) \in \mathbb{R}^{n+k}} \{(x, w) : a^\top x + b^\top w \geq h_D(a, b)\}.$$

Proof. The defining minimum makes the inequality valid at each point of G_D ; linearity preserves it under convex combinations. The graph is compact by continuity, and its convex hull is compact in finite dimension. A point outside that hull is strictly separated by a linear functional. Orient that functional as a lower support inequality to obtain a member of the displayed intersection that excludes the point. $\square$

This is classical convex hull duality; simultaneous graph separation from linear combinations is developed explicitly in [21, 26]. The computational problem is to evaluate or bound (1), and to find a useful direction without spending more than its benefit. A numerical direction generator is permitted to be inaccurate: its output only chooses (a, b) . Soundness depends on a bound for that *final* direction, not on the correctness or optimality of the direction search. The same observation allows random samples, LP direction search, and a native sampling kernel outside the cut-validity calculation.

For $n = 1$ and $k = 1$, optimizing over the coefficient of x recovers the biconjugate description of a scalar convex envelope. For $k > 1$, joint directions retain dependencies between outputs that intersected scalar envelopes can omit. A finite collection of valid joint cuts generally gives an outer approximation; absence of a violated generated cut does not establish membership in H_D .

Proposition 2.2 (Final floating-point row). *Suppose the exact values of the coefficients submitted to a numerical solver are $\hat{a} \in \mathbb{Q}^n$ and $\hat{b} \in \mathbb{Q}^k$ (in particular, finite binary floating-point numbers interpreted exactly). If a checker establishes*

$$L \leq \min_{x \in D} \{\hat{a}^\top x + \hat{b}^\top F(x)\}$$

and the submitted right-hand side $\hat{L}$ satisfies $\hat{L} \leq L$ exactly, the submitted mathematical row is valid on H_D .

Proof. Apply Proposition 2.1 to the submitted coefficients and then weaken the right-hand side from L to $\hat{L}$. $\square$

Rounding an already proved rational row without checking its resulting coefficients would not satisfy this contract. One alternative, when a trusted box $\ell \leq z \leq u$ is available for all lifted coordinates, is to replace the exact row $c^\top z \geq L$ by

$$\hat{c}^\top z \geq L + \sum_i \min\{(\hat{c}_i - c_i)\ell_i, (\hat{c}_i - c_i)u_i\},$$

and round the right-hand side downward. The formula follows by bounding each coordinate of $(\hat{c} - c)^\top z$. Our principal contract is the direct check for the final direction in Proposition 2.2; it avoids needing auxiliary-variable bounds for this correction.

3 Finite certificates on the whole domain

3.1 Exact Bernstein bounds for rational polynomials

Consider a polynomial p on a rational box $B = \prod_{i=1}^n [\ell_i, u_i]$. Substitute $x_i = \ell_i + (u_i - \ell_i)t_i$ and eliminate fixed coordinates. In the remaining coordinates, write

$$p(x(t)) = \sum_{\beta \leq d} a_\beta t^\beta = \sum_{\alpha \leq d} b_\alpha B_\alpha^d(t), \quad B_\alpha^d(t) = \prod_i \binom{d_i}{\alpha_i} t_i^{\alpha_i} (1 - t_i)^{d_i - \alpha_i}.$$

Here d is a componentwise degree bound and all coefficients are exact rationals. The conversion is

$$b_\alpha = \sum_{\beta \leq \alpha} a_\beta \prod_i \frac{\binom{\alpha_i}{\beta_i}}{\binom{d_i}{\beta_i}}. \quad (3)$$

Lemma 3.1 (Bernstein enclosure). *On B , $\min_\alpha b_\alpha \leq p \leq \max_\alpha b_\alpha$.*

Proof. For $0 \leq t_i \leq 1$, every $B_\alpha^d(t)$ is nonnegative and $\sum_\alpha B_\alpha^d(t) = 1$, by the binomial theorem in each coordinate. Thus $p(x(t))$ is a convex combination of the coefficients. Identity (3) follows by expressing each monomial t^β in the tensor-product Bernstein basis; the univariate identity has coefficients $\binom{\alpha}{\beta} / \binom{d}{\beta}$. $\square$

The enclosure is classical [11, 10]. The implementation contribution is to bind the exact polynomial, its original box, and the checked final row to the recorded partition and lower bounds. Adaptive subdivision can improve a weak enclosure without changing the validity argument.

Proposition 3.2 (Partition certificate). *Let $B_1, \dots, B_N$ be finitely many closed boxes whose union is B , and suppose $L_j \leq \inf_{B_j} p$ is verified for each j . Then $L = \min_j L_j \leq \inf_B p$. The same conclusion holds for a finite interval partition and any expression with a verified interval enclosure on every cell.*

Proof. Every $x \in B$ belongs to a checked cell B_j , where $p(x) \geq L_j \geq \min_i L_i$. $\square$

The coverage premise is essential. Checking all supplied cells while accepting an omitted end interval would not prove the claimed bound. The certificate checker must start from the trusted original domain and verify the entire subdivision, including endpoints. A cache key also needs the expression, direction, and domain: reusing a valid bound from a smaller domain on a larger domain is unsound.

3.2 Supported elementary expressions and explicit failure

For a supported elementary expression, outward ball or interval arithmetic supplies an enclosure for the support objective on each interval. Rational constants and final binary coefficients are embedded exactly. Expression evaluation must retain domain requirements: a logarithm needs a positive argument; division needs a denominator excluding zero; powers require their stated real domain. Algebraic simplification is not permission to remove the original expression's domain restrictions.

If a pole, invalid domain, nonfinite enclosure, unsupported operation, or resource limit prevents a complete bound, the procedure returns that status. An exhausted direction search means only that no checked useful cut was obtained within its budget. It does not prove that a point lies in a hull, that the original constraints hold, or that the domain is empty. Even a successfully checked cut need not be violated enough to justify insertion into the LP.

The elementary support calculation can also use a curvature correction. If the scalar direction p is twice continuously differentiable on $[a, b]$ and a verified bound gives $p'' \leq M$, then

$$\min_{[a,b]} p \geq \min\{p(a), p(b)\} - \max\{0, M\} (b-a)^2/8. \quad (4)$$

Indeed, $p(t) - Mt^2/2$ is concave, so its chord inequality rearranges to $p(t) \geq \text{chord}_p(t) - M(t-a)(b-t)/2$. Use $(t-a)(b-t) \leq (b-a)^2/4$ when $M \geq 0$; for $M < 0$, the chord itself is a lower bound. Verified lower endpoint values can replace $p(a)$ and $p(b)$. The implementation first checks all feature domains, including zero-weight features, then uses the maximum of this bound and available natural enclosures. An unavailable derivative bound or singular derivative disables this optional strengthening; it does not license removing a domain endpoint.

The supported grammar, actual certificate schema, and solver binding are stated with their implementation in Section 9. Neither Python's rational arithmetic nor the interval library is formally verified here. Replay reduces the trusted computational path and catches mismatched artifacts; it is not a proof of the underlying software stack.

4 Joint cuts in the original variables

Let the original model contain the inequality sides

$$g_i(x) + \ell_i(v) \leq b_i, \quad i = 1, \dots, m, \quad (5)$$

where x is a selected subset of original variables, v contains model variables, and each ℓ_i is linear. A variable may occur both in x and in an affine remainder. All constants are included in b_i . The block domain D contains the block coordinates of every feasible point to which a proposed cut will apply.

Proposition 4.1 (Original-variable support cut). *Choose $a \in \mathbb{R}^{\dim x}$ and $\lambda \in \mathbb{R}_+^m$. If*

$$\beta \leq \inf_{u \in D} \left\{ a^\top u + \sum_i \lambda_i g_i(u) \right\},$$

then the original-variable linear inequality

$$a^\top x - \sum_i \lambda_i \ell_i(v) \geq \beta - \sum_i \lambda_i b_i \quad (6)$$

is valid for the model.

Proof. At a feasible point, nonnegativity of the multipliers gives $\sum_i \lambda_i g_i(x) \leq \sum_i \lambda_i (b_i - \ell_i(v))$. Substitute this upper bound into the support inequality and rearrange. $\square$

This is weak Lagrangian duality. It needs neither convex functions nor optimal multipliers. The functions are minimized jointly before elimination; simply combining already linear valid inequalities cannot strengthen their intersection. Equality sides can use unrestricted multipliers, or two opposite inequality sides with nonnegative multipliers. A minimization objective $g_0(x) + \ell_0(v) + c_0$ supplies an epigraph side $g_0(x) + \ell_0(v) - t \leq -c_0$. A maximization objective uses the hypograph side $-g_0(x) - \ell_0(v) + t \leq c_0$. The objective transformation, constant, and sense must be identical in every compared mode. These rows do not require a separate equality variable for each selected nonlinear expression.

Theorem 4.2 (Closure of the direct-row interface). *Suppose D is nonempty and compact and $g : D \rightarrow \mathbb{R}^m$ is continuous. Put*

$$K = \text{conv}\{(x, g(x)) : x \in D\}, \quad P = \{(x, z) : \exists y, (x, y) \in K, y + Lz \leq b\}.$$

Then P is exactly the intersection, over all $a \in \mathbb{R}^{\dim x}$ and $\lambda \in \mathbb{R}_+^m$, of

$$a^\top x - \lambda^\top Lz \geq \min_{u \in D} \{a^\top u + \lambda^\top g(u)\} - \lambda^\top b. \quad (7)$$

If continuous equality features $h : D \rightarrow \mathbb{R}^r$ are included in the joint graph, their support coefficients are free and their target values are substituted without adding cone coordinates.

Proof. The set $U = K + (\{0\} \times \mathbb{R}_+^m)$ is closed and convex. To see closedness, take a convergent sequence of sums, extract a convergent subsequence of the compact K summands, and subtract; the cone summands then converge inside their closed cone. Membership in P is equivalent to $(x, b - Lz) \in U$.

A lower support functional of U with finite constant must have nonnegative coefficients on the last m coordinates, because a negative coefficient has infimum $-\infty$ along the corresponding cone ray. For the remaining directions its support value is $\min_{u \in D} (a^\top u + \lambda^\top g(u))$. Substituting $(x, b - Lz)$ gives (7). Closed convex separation excludes every point outside U by such a finite support inequality, proving the converse. For continuous equality features the joint graph $\text{conv}\{(x, g(x), h(x)) : x \in D\}$ is also compact. The same argument adds no cone in their coordinates, so their coefficients are free. $\square$

This describes the projection of the specified graph relaxation, not necessarily the convex hull of original feasible points. For example, on $[0, 1]$ the equality $x^2 = 1/4$ has feasible set $\{1/2\}$. The mixture with mass $3/4$ at zero and $1/4$ at one instead has $\mathbb{E}x = \mathbb{E}x^2 = 1/4$. The projected graph relaxation thus contains $x = 1/4$, and every direct support cut accepts it. Including $x^2 = 1/4$ inside the support domain changes the oracle problem and removes this example. Convexifying a graph and imposing constraints in expectation do not generally commute. Ordinary domain propagation can also resolve this particular equality; that does not change the comparison of cut families over the unchanged box.

Proposition 4.3 (Safe export after elimination). *Let the exactly eliminated row be $c^\top v \geq r$ and let $\hat{c}$ denote the exact rational values of the coefficients actually exported in binary64. With $\Delta_j = \hat{c}_j - c_j$, suppose valid global bounds imply a finite value*

$$E = \sum_j \inf_{L_j \leq s \leq U_j} \Delta_j s.$$

Then every exported lower side $\hat{r} \leq r + E$ gives a valid row $\hat{c}^\top v \geq \hat{r}$.

Proof. $\hat{c}^\top v = c^\top v + \sum_j \Delta_j v_j \geq r + E.$ □

A positive Δ_j needs a finite lower bound, a negative one a finite upper bound, and a zero one contributes zero even on an unbounded coordinate. This covers an unbounded objective epigraph when its exported coefficient is exact. The correction must follow coefficient merging, normalization, and any zero dropping. The lower side is rounded downward and checked by exact rational comparison. An unavailable finite correction, overflow, or lost useful separation causes refusal; a fixed safety shift cannot replace the calculation. The source sides, their orientations, multipliers, domain provenance, support bound, exact eliminated row, and actual exported row are separate parts of the certificate contract.

The derivation and its checked implementation are detailed in [theory/row-aggregation.md](#). The formulation and safe-rounding principles have direct prior literature; Section 11 records the sources and contribution boundary.

5 Exact quadratic support on a bounded polytope

Let $P = \{x \in \mathbb{R}^d : Ax \leq b\}$ be a bounded rational polytope and write a rational quadratic as

$$q(x) = \frac{1}{2}x^\top Hx + c^\top x + c_0, \quad H = H^\top.$$

The support of any vector of quadratic features has this form. The matrix H may be indefinite or singular. Finite supplied variable bounds make boundedness part of the checked input; equalities are represented by their two inequality sides.

For each subset S of at most d rows, form the stationary-face system

$$\begin{pmatrix} H & A_S^\top \\ A_S & 0 \end{pmatrix} \begin{pmatrix} x \\ \lambda \end{pmatrix} = \begin{pmatrix} -c \\ b_S \end{pmatrix}. \quad (8)$$

Retain its unique solution if the matrix is nonsingular and $Ax \leq b$ holds exactly. Multipliers are unrestricted: this is stationarity on an affine face, and sign filtering is unnecessary.

Theorem 5.1 (Complete finite quadratic support). *If P is nonempty, at least one retained candidate minimizes q globally. The least candidate value is therefore the exact rational support value, attained at a rational point. If no candidate is retained after the complete enumeration, P is empty. These statements include lower-dimensional polytopes, implicit equalities, redundant rows, and constant objectives.*

Proof. Compactness gives a global minimizer. Among global minimizers choose x_* whose minimal containing face F has the smallest dimension. It lies in the relative interior of F . Let T be the tangent space of the affine hull of F . Relative first- and second-order conditions imply $v^\top(Hx_* + c) = 0$ and $v^\top H v \geq 0$ for every $v \in T$.

If the Hessian restricted to T were singular on a positive-dimensional face, there would be a nonzero $v \in T$ with $v^\top H v = 0$. The quadratic identity and stationarity give $q(x_* + tv) = q(x_*)$ for every real t . The line meets the bounded face in a closed interval with x_* in its relative interior. At an endpoint it reaches a proper face with the same minimum, contradicting the choice of F . Thus H is positive definite on T unless $T = \{0\}$.

Choose an independent basis S of the active rows at x_* . It has at most d rows and $\ker A_S = T$. In a homogeneous solution (v, μ) of (8), $A_S v = 0$ and $Hv + A_S^\top \mu = 0$. Multiplication by $v^\top$ gives $v^\top H v = 0$, hence $v = 0$ by positive definiteness on T . Independence of the rows then gives $\mu = 0$.

The matrix is nonsingular, including the vertex case $T = \{0\}$. Relative stationarity places $Hx_* + c$ in the row span of A_S , so its unique solution includes x_* . The complete enumeration therefore retains a minimizer. All systems and evaluations are rational. Conversely, every retained candidate is feasible, so an empty polytope retains none. $\square$

If there are m rows including variable bounds, there are at most $N = \sum_{k=0}^d \binom{m}{k}$ subsets. Direct exact elimination on systems of order at most $2d$ and feasibility testing against all rows uses $O(N(d^3 + md))$ rational arithmetic operations. Rational determinant bounds control the encoding lengths of candidates and objective values. For each fixed d the bit complexity is polynomial in the input length; when d is part of the input, the subset count can be exponential. This is a classical face-enumeration algorithm [24, Section 2.9], not a new tractability result for unrestricted quadratic programming.

The implementation `theory/quadratic_polytope.py` checks its enumeration budget before claiming completion. A resource refusal is not an optimum or an emptiness certificate. Its certificate binds bounds, rows, and coefficients and records the complete candidate calculation. Replay reconstructs that calculation from trusted inputs; it shares exact arithmetic routines with generation. Independent tests and mathematical review supply different checks, but not formal verification of the Python interpreter or rational-arithmetic library.

The previous two-variable polygon algorithm is a more specialized version: it checks vertices, stationary edge points, and a positive-definite interior stationary point in $O(m^3)$ arithmetic operations. The new oracle permits arbitrary affine coupling in every admitted small dimension. It can thus merge overlapping quadratic blocks with leaf-leaf rows or products when the resulting block fits its budget. The next section retains a different advantage: constrained stars remain polynomial in their number of leaves without fixing that dimension.

6 A larger exact class: quadratic stars with affine rows

The obstruction in Section 7 motivates support directions that span adjacent pairs. A tractable extension is a center variable y with leaves $x_1, \dots, x_k$. The objective is

$$q(y, x) = a_0 + a_1 y + a_2 y^2 + \sum_{i=1}^k \{d_i x_i^2 + (e_i y + f_i) x_i\}. \quad (9)$$

All objective coefficients, affine-row coefficients, right-hand sides, and finite variable bounds are rational; finite binary floating-point inputs mean their exact rational values. Each supplied affine row contains only y and at most one leaf. Rows containing only y are allowed; rows involving two different leaves are outside this class. Every quadratic coefficient can have either sign. The same structure must hold for the scalar support objective obtained from a proposed direction.

For fixed y , each leaf is constrained to an interval

$$L_i(y) \leq x_i \leq U_i(y), \quad L_i(y) = \max_{r \in \mathcal{L}_i} (\alpha_{ir} + \beta_{ir} y), \quad U_i(y) = \min_{r \in \mathcal{U}_i} (\gamma_{ir} + \delta_{ir} y). \quad (10)$$

The original bounds provide at least one function in each envelope. Projecting each bounded center-leaf polygon onto the center gives a closed rational interval. Their intersection, together with the center bounds and center-only rows, is exactly the feasible center interval I : conditional leaf feasibility is independent across leaves. An empty intersection proves this support domain empty.

Theorem 6.1 (Exact rational star support). *For the nonempty domain just described, the minimum of (9) is rational and has a rational minimizer. It is obtained by a finite rational partition of I on*

which each leaf has an affine minimizer and the conditional optimum is quadratic in y . Both the partition and the attained minimum can be reconstructed exactly from the supplied rows, bounds, and coefficients.

Proof. Partition I at all intersections of affine functions in the envelopes of each leaf. On every open resulting interval, the active functions L_i and U_i are fixed and affine.

If $d_i > 0$, the unconstrained leaf minimizer is the affine function $v_i(y) = -(e_i y + f_i)/(2d_i)$. The constrained minimizer is $\max\{L_i(y), \min\{v_i(y), U_i(y)\}\}$. Further partition at intersections of v_i with its two active bounds. Each remaining interval has one affine minimizing rule.

If $d_i \leq 0$, a minimizing leaf value is an endpoint. The difference between the upper and lower endpoint objectives factors as

$$q_i(y, U_i(y)) - q_i(y, L_i(y)) = (U_i(y) - L_i(y))\{d_i(U_i(y) + L_i(y)) + e_i y + f_i\}.$$

The first factor is nonnegative on the feasible center interval. The second is affine on the current interval, so at most one additional rational root determines the preferred endpoint; an identically zero factor permits either endpoint. This also gives an affine minimizing rule after refinement.

Take the union of all leaf breakpoints. Substituting the selected affine rules into (9) gives a rational quadratic on every closed piece. The rule remains feasible and minimizing at its endpoints: bound switches coincide there, and endpoint-choice switches have equal values there. Minimize each quadratic at its endpoints and, for a positive square coefficient, its feasible stationary point. These candidates are rational and exhaustive. The selected center and leaf rules supply an attained rational minimizer. A singleton center interval uses the same conditional scalar minimization at that point. $\square$

If leaf i has m_i envelope lines, their pair intersections produce at most $O(m_i^2)$ center breakpoints; a constant number of further switches per envelope interval gives the same order. The union of breakpoints has polynomial size. The present implementation deliberately uses straightforward pair enumeration and scanning. Its arithmetic cost is polynomial in the supplied row counts and number of leaves; no claim of an optimal complexity bound is needed for this small-block oracle. Unlike general elimination on a deeper graph, this construction does not repeatedly nest piecewise messages along a tree.

The inherited implementation `theory/quadratic_star.py` records the complete center partition, affine leaf rules, quadratic value functions, and attained minima. Unsupported mixed terms or rows raise an explicit error instead of being dropped. Replay rebuilds the same exact object from trusted input. As with the polytope oracle, this is exact support for a supplied direction; completeness of the integration's direction search is a separate question.

Corollary 6.2 (Sharp gain for specified pair directions). *Let compact rational center-leaf polygons P_i assemble into a nonempty star, and fix rational quadratic pair objectives $q_i(y, x_i)$. Define*

$$\beta_i = \min_{P_i} q_i, \quad \beta_* = \min_{(y, x_i) \in P_i \ \forall i} \sum_i q_i(y, x_i), \quad \Delta = \beta_* - \sum_i \beta_i.$$

Then $\Delta \geq 0$ is the largest possible constant improvement in the sum of these fixed pair inequalities. It is exactly computable by the pair and star support algorithms. Moreover, $\Delta > 0$ if and only if the center projections of the pair minimizing sets have no common point.

Proof. Every pair objective is at least its individual minimum, proving $\Delta \geq 0$. Equality holds precisely when one feasible assembled point minimizes every pair objective. Such a point exists precisely when their minimizing center sets intersect, since leaves can then be chosen independently.

If no such point exists, compactness makes the attained assembled minimum strictly larger. The attained value β_* is the largest valid constant in this summed direction. $\square$

This gives a precise diagnostic for a proposed merger of directions. It does not choose the normals or predict whether computing the gain will improve total runtime. In the example below, both pair minima are zero and the exact gain is $1/128$.

The box-only case is a specialization of Del Pia and Khajavirad’s forest-QP algorithm, which gives an $O(n^2)$ arithmetic bound for general forests [25]. Piecewise-affine responses of strictly convex parametric quadratic programs are also classical [3]. The additional domain handled explicitly here consists of center-leaf affine rows with all signs of scalar leaf curvature. That scope distinction and its exact implementation are documented; a first-result claim does not follow from a bounded source search.

7 What overlapping pair hulls miss

Exact pair support does not make intersected pair hulls the exact global hull. The failure already occurs on a path x – y – z in the unit cube, even if every square and edge product is present. Define

$$H_{xy} = \text{conv}\{(x, y, x^2, xy, y^2) : 0 \leq x, y \leq 1\}$$

and H_{yz} analogously. Let R intersect these two hulls while identifying the shared first and second moments of y . The full sparse quadratic hull is

$$H = \text{conv}\{(x, y, z, x^2, y^2, z^2, xy, yz) : 0 \leq x, y, z \leq 1\}.$$

Proposition 7.1 (A strict pair-hull gap). *The point*

$$\bar{v} = (1/2, 1/2, 4/5, 1/2, 5/16, 4/5, 3/8, 1/2) \tag{11}$$

belongs to R but not H . The quadratic function

$$D = (y - 1/4 - x/2)^2 + (y - 5z/8)^2 + x(1 - x) + z(1 - z) \tag{12}$$

has minimum $1/128$ on the cube, whereas its lifted minimum over R is zero.

Proof. The left pair measure assigns probability $1/2$ to each of $(x, y) = (0, 1/4), (1, 3/4)$. The right pair measure assigns probabilities $1/5, 4/5$ to $(y, z) = (0, 0), (5/8, 1)$. Both give $\mathbb{E}y = 1/2$ and $\mathbb{E}y^2 = 5/16$, and their remaining moments yield (11). Thus the point belongs to R .

In a global representing measure, $\mathbb{E}[x(1 - x)] = \mathbb{E}[z(1 - z)] = 0$ would force x, z to be binary almost surely. The given moments also force $\mathbb{E}(y - 1/4 - x/2)^2 = \mathbb{E}(y - 5z/8)^2 = 0$. The first identity requires $y \in \{1/4, 3/4\}$, and the second $y \in \{0, 5/8\}$, almost surely. Their supports are disjoint, so no global measure exists.

For fixed y , the square coefficients of x and z in D are $-3/4$ and $-39/64$. A minimum therefore has binary leaves. At those endpoints the last two terms vanish. Set $a = 1/4 + x/2$ and $b = 5z/8$. Then

$$(y - a)^2 + (y - b)^2 = 2(y - (a + b)/2)^2 + (a - b)^2/2.$$

The midpoint lies in $[0, 1]$. The smallest difference between $\{1/4, 3/4\}$ and $\{0, 5/8\}$ is $1/8$, giving $1/128$, attained at $(x, y, z) = (1, 11/16, 1)$. Each pair summand of D is nonnegative on its pair box, and both have zero expectation under the local witnesses. Their lifted minimum over R is therefore zero. $\square$

Expanding $D \geq 1/128$ gives a cut involving only the existing sparse quadratic coordinates. With m_u, s_u, c_{uv} denoting the first, square, and product coordinates, it is

$$2s_y - \frac{1}{2}m_y - c_{xy} - \frac{5}{4}c_{yz} + \frac{5}{4}m_x - \frac{3}{4}s_x + m_z - \frac{39}{64}s_z \geq -\frac{7}{128}. \quad (13)$$

It excludes (11) by $1/128$, with no xz product. The star oracle of Theorem 6.1 computes this support value exactly. Scaling the objective scales the gap; $1/128$ is an exact diagnostic value, not a universal limit on missing strength.

This comparison is against exact *pair* hulls, not against a fully coupled SDP relaxation. A dense moment matrix together with the missing nonedge product and its McCormick inequalities excludes this witness too. At the proposed moments, $\text{Var}(y) = 1/16$, $\text{Var}(x) = 1/4$, $\text{Var}(z) = 4/25$, and the two edge covariances saturate the positive Cauchy–Schwarz bounds. A global PSD moment matrix would therefore require $\text{Cov}(x, z) = 1/5$, hence $\mathbb{E}[xz] = 3/5 > \mathbb{E}[x] = 1/2$, contrary to $xz \leq x$. The star cut repairs this example without adding that nonedge coordinate; general dominance over SDP or RLT is not asserted.

There is a simple family behind this example. Suppose $a, a + b, c, c + d \in [\ell, u]$ and let $\delta = \min\{|s - t| : s \in \{a, a + b\}, t \in \{c, c + d\}\}$. For $0 \leq x, z \leq 1$, $\ell \leq y \leq u$,

$$(y - a - bx)^2 + (y - c - dz)^2 + b^2x(1 - x) + d^2z(1 - z) \geq \delta^2/2. \quad (14)$$

The left side is affine in each leaf after expansion, so its minimum occurs at binary leaves. The preceding midpoint calculation proves both validity and sharpness. Automatic selection of the four parameters is not supplied merely by this family; the star support algorithm is the implemented general mechanism within its stated class.

7.1 The stronger interface that makes tree gluing valid

The obstruction concerns inconsistent representing distributions, not an error in the individual hulls. Under a running-intersection tree of compact block domains, local probability measures glue if they have the *same complete marginal distribution* on every adjacent separator [20]. Root the tree; retain the root measure and attach each child’s new variables using its conditional distribution given the parent separator. Matching marginals preserve that child’s distribution, and running intersection prevents inconsistent assignments of previously introduced variables. Induction constructs a global representing measure. Conditional distributions exist for these compact Euclidean Borel spaces; finite-support gluing is ordinary conditional probability and finite summation.

For a binary scalar separator, its mean determines the complete distribution. For a separator known to lie in a fixed set of k distinct scalar values, matching moments of orders $0, \dots, k - 1$ determines atom probabilities through an invertible Vandermonde system. Neither observation applies to unrestricted continuous separators with only first and second moments. Means of several binary separator variables also do not determine their joint distribution.

The counterexample does not contradict exactness of the McCormick relaxation for a *pure bilinear* graph on a forest. On the unit box, an edge with means m_i, m_j and product w_{ij} has binary probabilities

$$p_{11} = w_{ij}, \quad p_{10} = m_i - w_{ij}, \quad p_{01} = m_j - w_{ij}, \quad p_{00} = 1 - m_i - m_j + w_{ij}.$$

The McCormick inequalities are their nonnegativity conditions. Adjacent edges share a binary marginal fixed by the common mean, so these measures glue along the forest. Prescribed square coordinates can forbid this binary choice, as they do in (11).

The companion [theory/overlap-review.md](#) also gives a nonlinear constrained example that survives separate block interval propagation and is repaired by one additional separator statistic. That example illustrates a different possible extension; it is not claimed to defeat global presolve or to be an automatic statistic-selection algorithm. The inherited implementation supplies the concrete alternative of selected larger star support cuts. The current exact polytope oracle also handles their union whenever it fits its explicit enumeration budget.

8 Complete separation to a positive tolerance

Exact support for a chosen direction does not ensure that a bounded numerical search finds the direction needed to separate a query. This section supplies a finite fallback with a complete answer contract. Its potentially large cost is part of the result.

Let P be a nonempty compact rational polytope, let $F : P \rightarrow \mathbb{R}^k$ be a vector of rational polynomials, let $q \in \mathbb{Q}^k$ be a query, and choose a positive rational tolerance ϵ . A designated subset J of coordinates may have nonnegative support coefficients. Define

$$C_J = \{c \in [-1, 1]^k : c_j \geq 0 \ (j \in J)\}, \quad U_J = \text{conv } F(P) + \text{cone}\{e_j : j \in J\}.$$

For $J = \emptyset$, this is ordinary graph-hull separation, provided the coordinate features of x are included in F when required. For nonempty J , the cone expresses inequality-side slack, as in Theorem 4.2. Set

$$\psi(c) = \min_{x \in P} c^\top (F(x) - q).$$

Lemma 8.1 (Distance and direction continuity). *The distance in feature coordinates satisfies*

$$\text{dist}_1(q, U_J) = \max_{c \in C_J} \psi(c).$$

If rational bounds $\ell_j \leq F_j(x) \leq u_j$ hold on P , put $R = \sum_j \max\{|\ell_j - q_j|, |u_j - q_j|\}$. Then $|\psi(c) - \psi(d)| \leq R\|c - d\|_\infty$ on C_J .

Proof. For any $z \in U_J$, any $c \in C_J$ satisfies $\psi(c) \leq c^\top (z - q) \leq \|z - q\|_1$, giving one inequality. The set U_J is nonempty, closed, and convex by the compact-plus-cone argument in Theorem 4.2. If its distance from q is positive, separate it from the open L_1 ball about q with that radius. Scale the separating functional to infinity norm one. Its finite lower support requires $c_j \geq 0$ on cone coordinates, and the support value of the ball yields the reverse inequality. At distance zero, $c = 0$ and the first inequality give equality.

For the continuity bound, every $x \in P$ satisfies $|(c - d)^\top (F(x) - q)| \leq R\|c - d\|_\infty$. Taking minima preserves the same uniform error in both directions. $\square$

The field's support/separation equivalence is classical. The familiar polynomial-time weak-oracle equivalence requires a specified encoded convex-body model, including its inner and outer radius information [13]. The elementary construction here uses no inner ball and works for lower-dimensional hulls. Its enumeration cost is not a replacement polynomial-time theorem.

Theorem 8.2 (Finite positive-tolerance answer). *Suppose each chosen normal $c \in C_J$ can be assigned a feasible $p_c \in P$ and a certified lower support bound L_c satisfying*

$$L_c \leq \min_{x \in P} c^\top F(x) \leq c^\top F(p_c) \leq L_c + \delta, \quad 0 \leq \delta < \epsilon.$$

Choose an integer $N \geq \max\{1, \lceil 2R/(\epsilon - \delta) \rceil\}$. Enumerate the Cartesian normal grid with coordinates $-1 + 2i/N$ for unrestricted components and i/N for components in J , where $i = 0, \dots, N$. If any $L_c > c^\top q$, return the valid rational cut $c^\top z \geq L_c$. Otherwise $\text{dist}_1(q, U_J) \leq \epsilon$.

Proof. A returned cut is valid by its lower support bound and nonnegative cone coefficients, and it strictly excludes q . If none is returned, every grid normal satisfies $\psi(c) \leq c^\top (F(p_c) - q) \leq \delta$. Every normal in C_J is within infinity distance at most $2/N$ of a grid point. Lemma 8.1 bounds its support gap by $\delta + 2R/N \leq \epsilon$, and the distance identity proves the claim. $\square$

For quadratic features, Theorem 5.1 supplies exact support and a rational minimizer, so $\delta = 0$. A general rational polynomial graph also admits a finite support approximation. Let $v_1, \dots, v_s$ be all vertices of P and let

$$D_\infty = \max_i \left(\max_{x \in P} x_i - \min_{x \in P} x_i \right), \quad L \geq \sum_{j=1}^k \sum_{i=1}^d \max_{x \text{ in supplied box}} |\partial_i F_j(x)|.$$

Rational interval polynomial evaluation supplies a finite L . For a positive integer M , enumerate

$$S_M = \left\{ \sum_{i=1}^s \frac{n_i}{M} v_i : n_i \in \mathbb{Z}_+, \sum_i n_i = M \right\}.$$

All its points are feasible, even on an affine equality domain of zero ambient volume. This avoids the failure of filtering a rectangular floating-point grid for exact equality feasibility.

Lemma 8.3 (Rational domain net). *Every $x \in P$ is within infinity distance $D_\infty(s-1)/M$ of a point of S_M . Consequently, for every $c \in C_J$,*

$$\min_{x \in S_M} c^\top F(x) - \delta_M \leq \min_{x \in P} c^\top F(x) \leq \min_{x \in S_M} c^\top F(x), \quad \delta_M = LD_\infty(s-1)/M.$$

Proof. Express $x = \sum_i \alpha_i v_i$ as a convex combination. For the first $s-1$ coefficients put $n_i = \lfloor M\alpha_i \rfloor$ and put the remaining mass in n_s . The difference from the resulting sample is $\sum_{i < s} (\alpha_i - n_i/M)(v_i - v_s)$, whose infinity norm is at most $D_\infty(s-1)/M$. The derivative bound implies that $c^\top F$ is L -Lipschitz for the infinity norm on the box. Apply that inequality to a minimizer and its sample. The reverse bound uses $S_M \subset P$. $\square$

Choose $M \geq \max\{1, \lceil 2LD_\infty(s-1)/\epsilon \rceil\}$. Then $\delta_M \leq \epsilon/2$ and Theorem 8.2 applies. A singleton or constant graph causes no division by zero. The domain enumeration has at most $\binom{M+s-1}{s-1}$ candidates; the normal grid has $(N+1)^k$ directions. The vertex enumeration and each exact support calculation have their own costs. These bounds show finite termination and explicitly expose the dependence on dimension, vertex count, coefficient size, and tolerance. They are generally too large for a solver callback.

The implemented API `solver/separation.py` uses numerical proposals and exact convex-combination checks first. Successful proposals may avoid either grid. A rational convex combination $z_* \in \text{conv } F(P)$ proves the ordinary near-hull result when $\|z_* - q\|_1 \leq \epsilon$; with cone coordinates the check is

$$\sum_{j \notin J} |z_{*j} - q_j| + \sum_{j \in J} \max\{z_{*j} - q_j, 0\} \leq \epsilon.$$

Numerical weights are clipped to be nonnegative and renormalized in exact arithmetic, and every graph sample is checked against the exact domain. An inaccurate LP can therefore fail to help but cannot itself authorize an incorrect geometric verdict.

There are four mathematical return statuses: `cut`, `within_tolerance`, `empty_domain`, and `unresolved`. The first carries an exact rational separating row; the second a distance certificate; the third an exact empty-polytope calculation. The fourth has no geometric implication. In complete

mode, the enumeration budgets are removed and the rational algorithm terminates for every admitted finite input. In bounded mode, an exhausted budget returns `unresolved`. Unsupported or malformed inputs are explicit input errors, not one of the successful mathematical statuses. The bounded API limits support-call counts and polynomial domain-net candidates. It does not itself impose a wall-clock limit or cap the initial vertex enumeration and each exact quadratic face enumeration. The deployed callback uses a separate support wrapper with explicit face-count caps.

A near-hull grid certificate needs only one feasible point per direction whose checked gap is at most δ , together with verified grid coverage and $\delta + 2R/N \leq \epsilon$. Replay need not trust the direction LP or its claim to have optimized those points. A cut, in contrast, needs a lower bound on the whole domain. These distinct evidence requirements are checked separately. An optional binary64 version is corrected using feature bounds; its certificate records whether rounding preserved strict separation. Exact separation can survive when no useful numerical row does.

For direct-row separation use $F = (x, g(x))$, the query $(\bar{x}, b - L\bar{z})$, and nonnegative normal coordinates for g . An exact returned row maps through Proposition 4.1. A `within_tolerance` result measures distance in these feature/slack coordinates to U_J ; it is not automatically the same distance in the original model variables to the projected feasible set. The practical solver loop uses its explicitly bounded proposal policy. The complete API establishes an available finite completion of this specified separation problem, not an assertion that every callback runs that fallback.

9 The implemented model, cut, and replay contract

The common model builder and the cut generator have different responsibilities. The builder preserves the admitted original model at the submitted expression boundary. A cut can then use only a conservative domain justified from that model. Declining a difficult block leaves the original native model available; it does not authorize changing the function, discarding part of a row, or declaring the point feasible for the nonlinear constraints.

9.1 Source arithmetic and native expression binding

Every loaded finite binary64 number is interpreted as its exact rational value. Ordinary polynomial assembly can nevertheless change intermediate values: the source coefficient in $(0.1x)^2$ is the exact square of the binary64 leaf, whereas multiplying its coefficient in binary64 rounds that square again. Likewise a native constraint constructor can move a constant into its sides with a rounded subtraction. Neither difference is accepted merely because it is small.

The builder first assembles the ordinary native expression, then compares its exact interpreted value with the source expression by symbolic subtraction and expansion. If they differ, it constructs an explicit generic expression DAG with separate constant children and checks again. This retains the original coefficient operations instead of folding them into changed coefficients. A remaining difference gives an explicit model-admission refusal. Ranged constraint sides and objective sense and constants are part of the same check. The comparison has no numerical tolerance.

This boundary is the submitted PySCIPOpt expression DAG interpreted over the reals with its exact binary coefficients. SCIP’s later simplification, presolve, LP arithmetic, and feasibility tolerances are outside this symbolic binding claim. A successful model build is not a proof of the complete solve.

9.2 Original affine implications and exact domains

Original affine rows can prove finite or one-sided bounds that are absent from the declared box. For a row $\sum_i a_i x_i \leq b$, the exact lower activity of all terms other than j implies

$$a_j x_j \leq b - \sum_{i \neq j} \min\{a_i \ell_i, a_i u_i\}. \quad (15)$$

Division by the nonzero a_j gives the corresponding upper or lower bound; a lower row side is handled after negation. Integer and binary semantics permit exact integer rounding where justified. Every recorded deduction identifies the original row, side, target variable, and previous and new bounds. Replay rebuilds the premises and recomputes the sequence. A budget stop leaves a valid prefix; it does not claim propagation closure. When exported, lower bounds round outward downward and upper bounds upward.

Proposition 9.1 (Affine-bound provenance). *Starting from valid declared and variable-type bounds, every bound obtained by a finite sequence of (15) and valid integer rounding contains all original feasible points.*

Proof. At a feasible point, every other term is at least its lower activity under valid prior bounds. Subtracting those bounds from the original row gives (15). Dividing with the correct sign preserves it; rounding to an integer endpoint preserves all integer solutions. Induction proves the statement for the complete recorded prefix. $\square$

The model builder also checks affine rows proportional to a source domain argument. For example, the equality $x - y = 1$ proves that $\log(x - y)$ has a positive argument even when both coordinates are unbounded. This argument-level implication does not invent finite coordinate bounds.

Domain traversal visits every original source node before simplification. A zero multiplier, cancelled quotient, or zero exponent cannot erase the domain required by its child expression. One-sided exact intervals preserve useful information: $x \geq 0$ proves $1 + x \geq 1$ without an upper bound. When these bounds and original affine arguments do not prove a required domain, an exact existential constraint preserves it:

required domain	additional constraint
$d \neq 0$	$du = 1, \quad u \in \mathbb{R}$
$d > 0$	$du = 1, \quad u \geq 0$
$d \geq 0$	$d \geq 0.$

In each of the first two rows, choosing $u = 1/d$ proves the forward projection and the product equality proves the converse; with $u \geq 0$ it implies $u > 0$ and $d > 0$. These are exact strict-domain descriptions, with no arbitrary positive epsilon. Witness variables can be unbounded. The argument expression is itself compiled and checked against the source. Shared witnesses are permitted only for the same argument and requirement, and every nested original occurrence is still visited.

All solver modes use the same guards. They preserve domains such as those hidden in $0 \log x$, x/x , $(\sqrt{x})^2$, or $(\log x)^0$. They can also change numerical difficulty and construction time; those costs are measured. A strict source domain can give an unattained infimum. Numerical optimality status and finite feasibility tolerances do not prove attainment on that open domain.

The admitted variable-power rule is $b(x)^{e(x)} = \exp(e(x) \log b(x))$ after positivity of $b(x)$ is proved from original bounds and affine rows. The transformation is local to that source node; a value such as $\log 2$ remains a logarithm node, not an approximated coefficient. If positivity is unproved,

the builder returns `unsupported_variable_power_domain`. It does not add a positive-base guard indiscriminately: doing so could remove valid zero-base or negative-base points under broader exponent semantics. Fixed exponents must fit the native power-node representation exactly.

The supported grammar and metadata are specified in [numerics/model-contract.md](#). Its separate admission artifact checks the seven previously refused application models before optimization. Admission demonstrates broadened applicability of this importer; performance and original-incumbent residuals are different evidence.

9.3 Block discovery and bounded practical separation

The current integration groups complete nonlinear remainders of original row sides. Exact additive affine terms are separated and retained in the elimination formula. Overlapping remainders can form a larger group up to the admitted block size; a nonlinear remainder involving unsupported variables is declined as a whole. Original affine sides supported on the chosen variables supply additional domain rows. Dropping a row from the support domain enlarges it and can weaken a bound; extraction must not change or drop terms from the nonlinear remainder multiplied by that row multiplier.

Discovery is lazy, so a model solved in presolve does not pay block-search cost. A numerical sample LP proposes a normal for the linear coordinates and nonnegative multipliers for consistently signed nonlinear sides. Exact support is then computed for the actual direction. Where available, its rational minimizer becomes a new sample for a bounded exchange. Sampling and numerical LP output choose work; they establish no lower bound by themselves. Exact polytope vertices and their centroid seed proposals on thin affine domains. Their conversion to floating-point samples is still only a numerical proposal, not a proof that the converted points remain exactly feasible.

The practical support wrapper admits general quadratic blocks up to dimension four under a complete-subset budget, then uses inherited supported polynomial, elementary, or constrained-star kernels where applicable. The default standalone wrapper allows 5,000 subsets. The solver callback sets a limit of 2,000 subsets per exact polytope calculation; an unmet limit yields no cut from that calculation. Supported higher-degree polynomial and elementary bounds remain sound without an exact support claim. Partial-domain features whose complete support domain cannot be safely covered are declined. If algebraic cancellation yields a polynomial extension, support on a larger box is valid for original feasible points, but a closeness certificate for that extension is not a membership certificate for the original partial graph.

Three solver modes use the common original-model builder:

- **baseline** supplies no additional aggregation separator;
- **all** uses the bounded search on all admitted blocks;
- **auto** uses a fixed structural admission rule and requires a larger final normalized violation before inserting a cut.

The mode name **all** means all admitted blocks subject to its caps, not all mathematical support directions. The exact finite completion API of Section 8 is available separately; callback time limits are not relabeled as a complete separation result.

The frozen **auto** rule admits quadratic blocks containing a signed quadratic not proved convex, and either a non-axis affine domain row or at least two distinct nonlinear source rows. Two support failures deactivate a block for the remaining callbacks. This is a heuristic spending decision, not a certificate that the block cannot help at a later point. Both cut modes permit at most 32 blocks, six nonlinear sides and 16 affine domain sides per block, three root callbacks, 12 total cuts, four cuts

per callback, 24 support calls, and three LP exchange attempts per block and callback. Callback work receives the smaller of one second and five percent of the requested total budget, checked between operations. Nonpreemptive work can overrun that allowance; the campaign’s hard process limit is a separate safeguard. The final violation divided by $\max\{1, \sum_j |\hat{c}_j|\}$ must exceed 10^{-5} in **all** and $5 \cdot 10^{-4}$ in **auto**. The corrected discovery routine checks the existing deadline between source rows, additive terms, and group steps. An expired allowance discards unfinished discovery and records **discovery_incomplete** and **budget_exhausted**; it produces neither a coverage claim nor a hull-membership conclusion. Terms are converted to polynomials using only their own symbols. Nonrational coefficients and oversized terms that cannot be converted remain in the nonlinear remainder, so an unsupported remainder is declined without changing the native row. The prospective experiment and the separate correction validation are distinguished below.

Each accepted cut follows the same sequence: prove support for the actual direction; reconstruct source-side elimination exactly; combine repeated original-variable coefficients; apply the bound correction of Proposition 4.3; inspect the actual stored row; and submit it as a global root cut. All domain rows and rounding bounds must be original global implications. No incumbent cutoff or local presolve bound is used as an unmarked global premise. The row-conversion implementation is conservative on unbounded coordinates: although Proposition 4.3 needs only the bound in the error’s direction, the frozen code requires both finite bounds whenever that coordinate has nonzero rounding error. Exact coefficients need no bounds. This narrower admission can reject a mathematically safe row; it cannot strengthen an unsafe one.

There is no nonlinear feature-equality reformulation that differs across modes. The common objective epigraph and any original-domain witnesses are recorded in every mode. The new comparison therefore needs no separate duplicate reformulation control. The earlier four-mode experiment retains its control because its formulation did change.

9.4 Replay and the trust base

A cut record binds the original serialized instance, signed source row identities, variable identifiers, source nonlinear expressions, proved affine bounds, chosen support domain, final direction, exact support certificate, source-side elimination, rounding correction, and actual binary64 row. Fresh-process replay must derive these expected objects from the archived original model. A stored explanation or producer/checker agreement without that binding would be insufficient.

Exact arithmetic and interval routines are shared between generation and replay. The checker is independent of the numerical direction LP and solver search, but is not a second formally verified arithmetic implementation. Separate analytic examples, alternative-oracle checks, domain adversaries, and deliberate certificate corruptions provide distinct implementation evidence. They complement the proofs rather than replace them. Numerical original-coordinate residual checks and reference-bound comparisons remain explicitly separate from cut validity.

10 Prospective evaluation and verification

10.1 The earlier result is retained

The first experiment completed 316 scheduled records. Its 24 selected application models included 20 admitted models; baseline and matched reformulation control solved 19, while both cut modes solved 18. Automatic selection reduced direction LPs from 1,073 to 323 and callback time from 5.59 to 3.05 seconds relative to unconditional activation, but did not recover the lost solve. On 13

synthetic cases, the control and cut modes solved all 13 while baseline solved 12, so the extra solve could already be explained by reformulation.

All 1,082 recorded added cuts passed the first campaign’s replay and all 270 returned incumbents passed its numerical residual checks. Eight construction-error records lacked model/cut logs; those missing logs were excluded from successful replay coverage and retained as unknown counts. These historical results justify investigating a different integration, not a favorable claim about its performance in advance.

10.2 Frozen new comparison

The new [prospective protocol](#) was fixed before primary outcomes. It excludes the prior convexification campaign names and selects 30 cached MINLPLib OSiL models, ten each from convex models, nonconvex continuous models, and nonconvex models with integer variables. Eligibility uses size limits and syntax parsing, not success of the new importer or cuts. Within each stratum, a fixed SHA-256 ordering selects models. The 422 eligible models have stratum counts 85, 208, and 129. “New holdout” refers to the named earlier convexification campaigns, not to every use of these public instances elsewhere in the repository.

Every selected failure remains in its denominator. The common builder is used in **baseline**, **all**, and **auto**. Primary full runs use seed zero, one SCIP thread, one BLAS/OpenMP thread, a 30-second soft total integration budget, and relative gap tolerance 10^{-4} . Model loading, analysis, building, cut generation, and solving are included in the integration budget; process imports and independent result checking are counted in outer wall time. The hard process limit for these runs is 45 seconds. Nonpreemptive calls can overshoot a soft budget and must be reported.

The fixed schedule comprises 90 new-holdout full runs, 30 historical diagnostic runs, 39 full mechanism runs, 105 root-only runs, and 18 seed-one repeats: 282 jobs. The 13 mechanisms use ten-second soft budgets; root-only runs use five seconds and one node. Mode order rotates by model index. One worker runs at a time on a host shared with unrelated jobs, so small timing differences remain descriptive. The 150-minute aggregate emergency cap exceeds the sum of all scheduled hard process timeouts; any unstarted jobs would still be recorded.

Frozen source copies and hashes, exact tagged parsed models, original OSiL files, mode configurations, solver status, numerical bounds, nodes, original-variable incumbents, work statistics, and cut witnesses are retained. Every returned incumbent is evaluated against original rows, bounds, domains, integrality, and objective by a separate evaluator at a scaled tolerance of 10^{-5} . Recorded root and final bounds are compared with known synthetic optima or archived feasible MINLPLib objectives. Such a conflict is a diagnostic flag, not by itself a proof about which numerical calculation is wrong. Fresh-process replay checks each saved cut against its original model and actual stored row, with tampering controls and explicit accounting for missing logs.

The campaign source snapshot is distinct from the final working tree. The first post-freeze changes rejected cut sides at SCIP’s infinity sentinel, checked that side in stored-row auditing, and caught failed heuristic exchange-sample evaluations. The campaign then exposed four worker errors in cut modes on **chp_partload** and **waterno2_06**: affine splitting passed an unnecessarily large list of symbolic generators to polynomial conversion. Discovery on a large case also overran its allowance without checking between rows. The repair uses each term’s actual generators, safely declines terms whose rational polynomial conversion is unavailable, and checks the existing work allowance between discovery operations. These repairs have their own targeted review and matched validation runs.

The archived prospective experiment keeps its source and every original result. Its timings describe that frozen implementation. Missing worker cut logs remain unknown. A separate

supplement repeats the affected cases with all comparison modes after the correction; it does not replace the original outcomes or turn outcome-selected repairs into a new holdout. The final reconciliation records exact repair selection, source identities, work-budget effects, and replay coverage separately.

10.3 Prospective campaign results

The campaign completed all 282 scheduled jobs in 1,338.6 seconds of outer wall time, with no hard process timeouts or unstarted jobs. There were 106 optimal statuses, 74 gap-limit statuses, 31 time limits, 67 node limits, and the four retained worker errors described above. All 30 new holdout models were admitted in every mode. The same 25 were numerically solved in every mode; the same five remained unsolved. Here solved means an optimal or gap-limit status with an incumbent passing the independent original-model numerical checks.

Mode	Solved/selected	Added cuts	Cases with cuts	Total seconds
Native baseline	25/30	0	0	170.57
All admitted blocks	25/30	38	7	187.02
Automatic selection	25/30	10	3	187.12

Total seconds sum the recorded integration and preparation time; they exclude interpreter imports and independent result checking. Shared-host timing is descriptive. The cut modes establish no solved-count gain in this population and take more summed time in these runs.

Automatic selection reduced candidate LPs from 186 to 66 and support calls from 138 to 51. It did not substantially reduce callback time: 27.63 seconds for `all`, versus 27.00 for `auto`. Discovery accounted for 24.41 and 24.63 seconds respectively, preceding most of the admission policy’s savings. Of 30 full runs per cut mode, 28 invoked discovery, 12 found no supported blocks, and two solved without calling this separator. The admitted groups contained 156 possibly overlapping blocks, of which 63 satisfied the structural automatic rule; 186 source sides were declined. These are observed coverage and work counts, not unique variables or a causal decomposition of native relaxation strength.

At the prespecified scaled dual-comparison tolerance 10^{-6} , final bounds for `all` were better on two models, tied on 21, and worse on seven; `auto` had two better, 24 tied, and four worse. Both improved bounds occurred on models already solved in all modes. Among the five common unsolved cases, both cut modes had zero better, one tied, and four worse bounds. None of those five received an added cut. For example, `graphpart_clique-40` finished with lower bound 438 for baseline and 392 for each cut mode. Discovery and callback cost, time limits, and shared-host variation prevent attributing every difference to one cause; these results do not show useful added strength on the common hard cases.

The root-only application comparison gave one better, 26 tied, and three worse bounds for `all`, versus one better, 28 tied, and one worse for `auto`. The seed-one repeats solved all six selected models in every mode and had six tied final bounds in each comparison. The complete root and repeat tables remain in [the campaign summary](#). The repetitions are too small to establish a population-wide timing effect.

Every mode solved 12 of the 13 full synthetic cases. A positive local mechanism is retained: on the root-only `simplex_quadratic_vector` case, both cut modes added two cuts and reached an optimal status with bound approximately -0.50000002 , versus baseline’s node-limit bound -0.50027374 . Its true optimum is $-1/2$. This is evidence that the row-domain support can supply

useful information on a constructed case; it is not an extra full-solve success on the application sample.

The ten historical diagnostics gave five solved cases in each mode. The four cut-mode failures occurred after model construction, so they must not be reported as importer refusals. The successful admission checks for the seven earlier refused models likewise establish importer coverage, not a guarantee that every subsequent separator operation finishes successfully. The repaired runs below address those errors and the discovery-budget defect directly.

Fresh-process replay passed all 123 recorded cuts, including their actual stored SCIP rows, and verified 156 frozen source/input hashes. All 14 tampering controls were rejected. The four error records have no complete cut logs and retain unknown counts; the successful replay claim excludes them. All 271 returned incumbents passed the independent numerical original-model checks, with no recorded reference-bound conflicts. The [replay record](#) and independent experiment audit provide the exact coverage. These results certify the recorded cut inequalities under the documented trust base, not the numerical solver’s final global bounds or complete search.

10.4 Matched repair validation

The correction cohort was fixed before any repaired optimization outcome. It includes every original model/phase/seed group whose cut mode exceeded its recorded discovery allowance or failed on the identified global-symbol recursion path. The rule selected 25 groups and 75 matched jobs, including all three modes for each group. The exact triggering values and original record hashes are retained in [the repair plan](#); the [repair protocol](#) keeps the original phase, seed, time and node limits, and mode order. The corrected code has a separate source snapshot. No activation threshold, instance-name filter, or selected population is retuned.

This is a regression cohort selected from observed failures and budget violations, not another prospective holdout. Its results remain separate and cannot be spliced into the 30-model table above. All 75 jobs completed in 754.5 seconds, with no worker errors, missing cut logs, hard timeouts, or unstarted jobs. The solved counts were identical across the three modes within each cohort:

Matched cohort	Groups	Solved per mode	All cuts	Auto cuts
Full application runs	8	4	12	3
Full historical diagnostics	7	4	10	0
Root application runs	9	0	11	3
Seed-one repeat	1	1	3	0

Every group has a baseline, an `all`, and an `auto` run; the table’s solved column applies to each mode, not their sum. Baseline adds no new cuts. There are no additional solves from either cut mode. For the eight affected full application models, final bounds were zero better, five tied, and three worse for `all`; `auto` had zero better, six tied, and two worse. Summed integration times were 123.91, 124.86, and 124.82 seconds for baseline, `all`, and `auto`. For the seven historical diagnostics both cut modes had six tied bounds and one worse bound, on `waterno2_06`. All root and seed-repeat bound comparisons tied at the stated tolerance.

The four formerly crashing diagnostic runs now stop incomplete discovery and continue native SCIP. Six discovery calls in the correction cohort crossed their soft deadline by a single nonpreemptive unit of work; all six reported incomplete discovery. The largest discovery excess was 3.618 milliseconds. The largest full-callback excess was 29.529 milliseconds, and the largest total soft-budget excess was 23.611 milliseconds. These measurements support the checked stopping contract, not a hard real-time bound or a claim that the discarded large blocks were successfully analyzed.

All 42 recorded repair cuts passed fresh-process original-model and actual-row replay, with 103 archived source/input hashes verified and all 14 tampering controls rejected. All 67 returned incumbents passed the independent numerical checks, with no reference-bound conflicts. The [repair results](#), [replay](#), and [independent experiment review](#) retain the original and repaired evidence separately. Across the two current campaigns, all 165 recorded cuts passed replay; the four original unknown logs remain outside that claim.

The operational recommendation remains native SCIP by default. The corrected implementation resolves the observed execution and discovery-budget defects, supplies optional certified cuts, and has no demonstrated general solve advantage in these experiments. The exact support and complete tolerance-separation APIs remain reusable mathematical components independent of that negative activation result.

10.5 Targeted implementation and document checks

The initial combined topic-specific local test command passed 250 tests and eight subtests. After the discovery repair, the focused integration, row arithmetic, and replay command passed 81 tests and three subtests. The [verification record](#) gives the actual command and preserves an earlier test-collection import failure and its correction. Separate independent support diagnostics checked 40 analytic and adversarial polytope cases, including equality domains, flat valleys, affine images of simplexes, and exhaustive Boolean MaxCut controls. The separation review checked 21 cases covering finite-net completion, missed interior normals, thin domains, coordinate cones, polynomial support error, tiny rational gaps, and altered certificates. The numerical direction LP is not an oracle for any of these checks.

The source/domain review includes cancelled restrictions, one-sided unbounded intervals, original affine implications, positive-base variable powers, generic-expression constant handling, and deliberate model-binding corruption. Integration replay tests reconstruct actual SCIP rows from original source sides and reject altered premises. Owner tests and internal independent reviews cover different risks; agreement of producer and replay alone would not establish the underlying theorem.

The report’s [document verification record](#) gives its targeted LaTeX build and evidence-link checks. No project-wide verification or CI inspection is part of these local records. Finite tests support implementation confidence under the proved contracts; they do not prove a whole numerical SCIP solve or a general performance advantage.

11 Prior results and contribution boundary

Simultaneous convexification and support of vector graphs precede this work. Tawarmalani develops inclusion certificates and simultaneous convexification [26]; Ballerstein’s thesis treats vector univariate graphs in a solver setting [2]; and Liers and coauthors develop simultaneous convexification for gas networks [21]. He and Tawarmalani’s composite and factorable relaxations already address multiple outer functions and shared-variable graph constructions [15, 16, 17]. The recent axis-aligned framework also treats simultaneous factorable graphs with feasible coupling [28]. The present use of a weighted sum is an implementation basis, not a new general convexification principle.

Low-dimensional quadratic hulls have exact conic and lifted representations [1]. Murty’s exposition of total face enumeration [24, Section 2.9] provides a direct precedent for active-face stationarity. Rational quadratic certificates are also classical; Vavasis proves membership of rational quadratic programming in NP [27]. The new support implementation makes a direct finite enumeration and its degenerate cases checkable. Box-constrained quadratic stars are already

inside the forest class of Del Pia and Khajavirad [25]; strictly convex parametric quadratic programs have classical piecewise-affine optimizers [3]. The inherited star construction states its additional center-leaf affine-domain and arbitrary-curvature cases explicitly. Neither its implementation nor a bounded source search proves publication priority. Sparse SDP and moment approaches provide further relevant exactness and overlap results [19, 20].

The aggregation formula is weak Lagrangian duality [6, Sections 5.1–5.2]. Quadratic aggregation hull results have substantial additional structure and hypotheses [8, 5]. Original-variable Lagrangian rows also occur in optimization-based bound tightening [12]. Those rows can be redundant for the generating LP yet useful for propagation. Here any additional strength must come from joint optimization of the nonlinear aggregate over its domain. Grouping small sets of quadratic terms before relaxation has established solver precedents as well [22].

Safe numerical cuts are established technology [7]. The bound correction after rational-row conversion in Proposition 4.3 is the lower-row form of the safe-approximation and aggregation principles of Eifler and Gleixner [9, Sections 2.2–2.3]. Validated polynomial underestimation has a direct Bernstein precedent [11, 10]. The current contribution is the supported expression and domain contract, its exact or outward arithmetic realization, the actual inserted-row binding, and the reproducible record of use.

Complete positive-tolerance separation is also an established type of problem. The classical weak optimization/separation equivalence [13] is stronger algorithmically under its stated encoded-body assumptions. Our finite normal and domain nets give a direct proof and implementation without a full-dimensionality assumption, at explicit and often impractical cost. The source audit also checked the relevant 1984 corrigendum’s scope [14]; it addresses the later submodular result, not the cited Theorem 3.1.

Native SCIP supplies substantial nonlinear relaxation and propagation machinery [4]. Its bivariate projected-domain relaxations [23], quadratic handling, and lifted cuts make a comparison against only scalar McCormick inequalities too weak to establish practical value. The experiments therefore compare complete native-model runs with the identical model builder and retain unfavorable outcomes.

The primary-source audit is [literature/README.md](#), with the exact aggregation precedents in [literature/aggregation-prior.md](#). Its source manifest records versions, inspected passages, retrieval limitations, and hashes. Inherited references not re-read in this continuation retain their earlier provenance. For example, the current Vavasis check covers the publisher abstract and metadata; the report’s finite candidate proof is given explicitly rather than attributed to an unread proof. The research package makes no publication-priority claim.

12 What completion does and does not imply

The exact support and finite separation contracts resolve specified mathematical problems. Their definitions also expose two limits that cannot be removed by more careful cut implementation alone.

The documented development program is complete: proofs and exact APIs cover the supported classes; admitted model semantics and exported rows are checked; the finite separation fallback has defined success and budget-stop meanings; and the prospective experiment plus matched repair validation are retained and reviewed. The deployment decision is also resolved by the evidence: keep native SCIP as the default and the extra separator optional. Equal application solve counts and repaired execution errors do not establish a general solver improvement.

Proposition 12.1 (A general exact-support hardness boundary). *Exact support of arbitrary rational quadratic graphs on boxes, with dimension as part of the input, is NP-hard.*

Proof. Let an undirected graph have nonnegative integer weights w_{ij} and put

$$q(x) = - \sum_{\{i,j\} \in E} w_{ij}(x_i + x_j - 2x_i x_j), \quad x \in [0, 1]^{|V|}.$$

The function is affine in each coordinate separately. Starting from a global minimizer, move one coordinate at a time to a minimizing endpoint while keeping the others fixed; its value cannot increase. A binary global minimizer therefore exists. At a binary point, $x_i + x_j - 2x_i x_j$ is one precisely when the edge crosses the associated cut. Thus $\min q = -\text{MaxCut}(G, w)$. The construction is polynomial in the encoded input, and weighted MAX CUT is NP-complete [18]. $\square$

Consequently an unrestricted polynomial-time exact support method would imply P=NP. The fixed-dimension and constrained-star support results do not conflict with this boundary: one fixes dimension, and the other restricts the interaction and constraint structure. The finite polynomial graph separator is complete precisely because its potentially exponential enumeration cost remains explicit. Hardness does not prohibit useful heuristics, special classes, approximations, or better algorithms; it prohibits treating unrestricted tractable support as a routine missing step under the usual complexity assumption.

Second, complete separation of the chosen graph relaxation does not complete the original feasible-set hull. The $x^2 = 1/4$ example in Section 4 proves that distinction even in one variable. Choosing stronger support domains or larger merged blocks can help; the cost and exactness of those choices are new questions, not consequences of finding every normal for the weaker relaxation.

There is also a practical distinction. A valid, complete, or stronger oracle need not improve a solver’s runtime. The native solver can already derive comparable information, an expensive direction can arrive too late, or the cuts can alter numerical behavior. Prospective comparisons can finish with a negative deployment decision. That is evidence about the investigated method, not a reason to omit its outcomes or rename unsuccessful work as a success.

Finally, these certificates cover their bound model domains and submitted rows. They do not verify all SCIP presolve substitutions, LP bases, node bounds, spatial branching, integer inference, or original-model feasibility and optimality in exact arithmetic. An end-to-end proof-producing MINLP solver would need those additional components. Internal reviews and replayed cuts establish the stated component-level evidence only.

References

- [1] Kurt M. Anstreicher and Samuel Burer. Computable representations for convex hulls of low-dimensional quadratic forms. *Mathematical Programming*, 124:33–43, 2010. Theorem locators in this report refer to the February 6, 2007 author preprint.
- [2] Martin Ballerstein. *Convex Relaxations for Mixed-Integer Nonlinear Programs*. PhD thesis, ETH Zurich, 2013. Chapter 5 details retained from the earlier repository audit; full PDF retrieval returned HTTP 429 in this continuation.
- [3] Alberto Bemporad, Manfred Morari, Vivek Dua, and Efstratios N. Pistikopoulos. The explicit linear quadratic regulator for constrained systems. *Automatica*, 38(1):3–20, 2002.
- [4] Ksenia Bestuzheva, Antonia Chmiela, Benjamin Müller, Felipe Serrano, Stefan Vigerske, and Fabian Wegscheider. Global optimization of mixed-integer nonlinear programs with SCIP 8. *Journal of Global Optimization*, 91(2):287–310, 2025. Online publication 2023; detailed section locators refer to arXiv:2301.00587v1, the longer preprint.

- [5] Grigoriy Blekherman, Santanu S. Dey, and Shengding Sun. Aggregations of quadratic inequalities and hidden hyperplane convexity. *SIAM Journal on Optimization*, 34(1):98–126, 2024.
- [6] Stephen Boyd and Lieven Vandenbergh. *Convex Optimization*. Cambridge University Press, 2004.
- [7] William Cook, Sanjeeb Dash, Ricardo Fukasawa, and Marcos Goycoolea. Numerically safe Gomory mixed-integer cuts. *INFORMS Journal on Computing*, 21(4):641–649, 2009.
- [8] Santanu S. Dey, Gonzalo Muñoz, and Felipe Serrano. On obtaining the convex hull of quadratic inequalities via aggregations. *SIAM Journal on Optimization*, 32(2):659–686, 2022.
- [9] Leon Eifler and Ambros Gleixner. Safe and verified Gomory mixed-integer cuts in a rational mixed-integer program framework. *SIAM Journal on Optimization*, 34(1):742–763, 2024. Theorem locators refer to version 2.
- [10] Jürgen Garloff, Christian Jansson, and Andrew Paul Smith. Inclusion isotonicity of convex-concave extensions for polynomials based on Bernstein expansion. *Computing*, 70(2):111–119, 2003.
- [11] Jürgen Garloff and Andrew P. Smith. Rigorous affine lower bound functions for multivariate polynomials and their use in global optimisation. In *Proceedings of the 1st International Conference on Applied Operational Research*, volume 1 of *Lecture Notes in Management Science*, pages 199–211, 2008. Also issued as Konstanzer Schriften in Mathematik und Informatik 250, June 2008.
- [12] Ambros M. Gleixner, Timo Berthold, Benjamin Müller, and Stefan Weltge. Three enhancements for optimization-based bound tightening. *Journal of Global Optimization*, 67:731–757, 2017. Theorem locators refer to the March 7, 2016 author manuscript.
- [13] Martin Grötschel, László Lovász, and Alexander Schrijver. The ellipsoid method and its consequences in combinatorial optimization. *Combinatorica*, 1(2):169–197, 1981. Theorem 3.1 and the convex-body assumptions on page 172 inspected.
- [14] Martin Grötschel, László Lovász, and Alexander Schrijver. Corrigendum to our paper “The ellipsoid method and its consequences in combinatorial optimization”. *Combinatorica*, 4(4):291–295, 1984. First-page correction scope inspected; concerns the submodular minimization result.
- [15] Taotao He and Mohit Tawarmalani. A new framework to relax composite functions in nonlinear programs. *Mathematical Programming*, 190:427–466, 2021.
- [16] Taotao He and Mohit Tawarmalani. Tractable relaxations of composite functions. *Mathematics of Operations Research*, 47(2):1110–1140, 2022.
- [17] Taotao He and Mohit Tawarmalani. MIP relaxations in factorable programming. *SIAM Journal on Optimization*, 34(3):2856–2882, 2024.
- [18] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. Original pp. 94, 97, and 100 inspected in the 2010 reprint; the PDF also contains a new introduction.

- [19] Aida Khajavirad. Tight semidefinite programming relaxations for sparse box-constrained quadratic programs, 2026. Version 2.
- [20] Jean B. Lasserre. Convergent SDP-relaxations in polynomial optimization with sparsity. *SIAM Journal on Optimization*, 17(3):822–843, 2006.
- [21] Frauke Liers, Alexander Martin, Maximilian Merkert, Nick Mertens, and Dennis Michaels. Solving mixed-integer nonlinear optimization problems using simultaneous convexification: a case study for gas networks. *Journal of Global Optimization*, 80(2):307–340, 2021.
- [22] Ruth Misener and Christodoulos A. Floudas. Global optimization of mixed-integer quadratically-constrained quadratic programs (MIQCQP) through piecewise-linear and edge-concave relaxations. *Mathematical Programming*, 136:155–182, 2012.
- [23] Benjamin Müller, Felipe Serrano, and Ambros Gleixner. Using two-dimensional projections for stronger separation and propagation of bilinear terms. *SIAM Journal on Optimization*, 30(2):1339–1365, 2020.
- [24] Katta G. Murty. *Linear Complementarity, Linear and Nonlinear Programming*. University of Michigan, internet edition, 1997. Original edition published by Heldermann Verlag, Berlin, in 1988. Section 2.9, pp. 163–166, inspected in the author’s chapter PDF.
- [25] Alberto Del Pia and Aida Khajavirad. Treewidth and the complexity of box-constrained quadratic programs, 2026. Version 1, September 28, 2026.
- [26] Mohit Tawarmalani. Inclusion certificates and simultaneous convexification of functions. Optimization Online manuscript, 2010.
- [27] Stephen A. Vavasis. Quadratic programming is in NP. *Information Processing Letters*, 36(2):73–77, 1990.
- [28] Haisheng Zhu, Taotao He, and Mohit Tawarmalani. Axis-aligned relaxations for mixed-integer nonlinear programming, 2026. Version 1 inspected.